

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «МОСКОВСКИЙ
АВИАЦИОННЫЙ ИНСТИТУТ (НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ)»

ОТЧЁТ
О ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЕ
по теме:
РЕАЛИЗАЦИЯ ГЛОБАЛЬНОГО ВТОРИЧНОГО ИНДЕКСА И
СРАВНИТЕЛЬНЫЙ АНАЛИЗ ЕГО ПРОИЗВОДИТЕЛЬНОСТИ С
ЛОКАЛЬНЫМИ ИНДЕКСАМИ В РАСПРЕДЕЛЕННОЙ СУБД PICODATA

Руководитель НИР,
Канд. техн. наук, доц.

подпись, дата

Романенков Д.В.

Исполнитель НИР,
Студент группы
М8О-403Б-22

подпись, дата

Клименко В.М.

РЕФЕРАТ

Отчёт 38 с., 1 рис., 1 табл., 15 ист.

РАСПРЕДЕЛЕННЫЕ СУБД, БАЗЫ ДАННЫХ, ВТОРИЧНЫЙ
ИНДЕКС, ГЛОБАЛЬНЫЙ ВТОРИЧНЫЙ ИНДЕКС, МАТЕМАТИЧЕСКОЕ
МОДЕЛИРОВАНИЕ, СИСТЕМЫ МАССОВОГО ОБСЛУЖИВАНИЯ

СОДЕРЖАНИЕ

Термины и определения	5
Перечень сокращений и обозначений	6
1 Вербальное описание области, направления работы, места и роли продукта, план работы	7
1.1 Распределенные системы управления базами данных	7
1.2 Системы массового обслуживания	7
1.3 Проблема поиска	8
1.4 Решение проблемы	9
1.5 Место и роль продукта ВКР	9
2 Описание предметной области	11
2.1 Верхнеуровневое описание СУБД Picodata	11
2.1.1 Сегментирование	12
2.2 Диаграмма «сущность-связь»	13
2.2.1 Описание условных обозначений	14
2.2.2 Описание сущностей	14
2.2.3 Описание связей	15
3 Математическая модель функционирования СУБД Picodata как СМО . .	17
3.1 СУБД Picodata как СМО	17
3.1.1 Показатели эффективности	19
3.2 Общие положения модели	20
3.3 Поиск в локальном индексе	23
3.3.1 Расчет показателей эффективностей	29
3.4 Поиск в глобальном индексе	30
3.4.1 Расчет показателей эффективностей	30
3.5 Обновление в локальном индексе	30
3.6 Обновление в глобальном индексе	31

3.7	Результат расчетов и сравнение	31
4	Модель функционирования плагина распределенного индекса	32
4.1	Контекстная диаграмма	32
4.2	Диаграмма потоков данных	32
4.2.1	Функциональная декомпозиция	32
5	Архитектура плагина распределенного индекса	33
5.1	Архитектура плагина	33
5.1.1	Диаграмма развертывания	33
5.1.2	Представление данных	33
5.1.3	Описание алгоритмов	33
6	Описание способов определения показателей качества ПО	34
6.1	Показатели качества	34
6.2	Способы определения показателей качества	34
7	Программа и методики испытаний ПО	35
7.1	Модель функционирования системы тестирования	35
7.1.1	Контекстная диаграмма	35
7.1.2	Диаграмма потоков данных	35
7.2	Архитектура системы тестирования индексных структур	35
7.2.1	Общие сведения	35
7.2.2	Структурные представления	35
7.2.3	Архитектура решения	35
7.3	Сравнение полученных результатов с математическими моделями	35
	Заключение	36
	Список использованных источников	37

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

В настоящем отчете о ВКР применяют следующие термины с соответствующими определениями.

Узел — один запущенный процесс базы данных

Мастер — узел, принимающий запросы на запись и чтение

Реплика — узел, принимающий запросы только на чтение

Набор реплик — узлы, хранящие одинаковую часть данных, среди которых один является мастером, а остальные являются репликами, применяющими поток изменений от мастера

Сегментирование — разбиение данных по наборам реплик

Бакет — единица балансировки. Виртуальная сущность, к которой однозначно привязаны данные

Таблица — совокупность связанных данных, хранящихся в структурированном виде

Первичный ключ — поля таблицы, по которым уникально идентифицируются записи

Вторичный ключ — поля таблицы, по которым неуникально идентифицируются записи

Ключ сегментирования — поля таблицы, по которым происходит определение ее бакета, чаще всего является первичным ключом

Сегментированная таблица — таблица, данные которой распределены между набором реплик

Кластер — сеть набора узлов, предоставляющий доступ к единой СУБД

Плагин — единица программного обеспечения, представленная динамической библиотекой, расширяющая функционал узла

Драйвер — программное обеспечение, предоставляющее интерфейс взаимодействия с кластером

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В настоящем отчете о ВКР применяют следующие сокращения и обозначения.

СУБД — система управления базами данных

ПО — программное обеспечение

ОЗУ — оперативное запоминающее устройство

ЦПУ — центральное процессорное устройство

ЛВИ — локальный вторичный индекс --- вторичный индекс, сегментирование которого зависит от ключа сегментирования индексируемой таблицы

ГВИ — глобальный вторичный индекс --- вторичный индекс, сегментирование которого зависит от вторичного ключа индексируемой таблицы

1 Вербальное описание области, направления работы, места и роли продукта, план работы

Для понимания прикладной области продукта ВКР, нужно рассмотреть две сущности: распределенные системы управления базами данных (СУБД) и системы массового обслуживания (СМО).

1.1 Распределенные системы управления базами данных

Единственные задачи СУБД как хранилища — обновлять пользовательские данные и в будущем искать среди них определенные записи. Для быстрого поиска в памяти подавляющего большинства СУБД поддерживается дополнительная структура данных, строимая над таблицей — индекс.

Индексы могут быть построены над двумя видами ключей:

- первичными ключами — набору полей, уникально идентифицирующими записи в таблице;
- вторичными ключами — набору полей, неуникально идентифицирующими записи в таблице.

В пункте 1.3 будет показано почему это разделение важно в рамках распределенных СУБД.

1.2 Системы массового обслуживания

Электронные СМО сталкиваются с вызовом эффективной обработки потока заявок, поступающих в случайные моменты времени. В ситуации, когда покупка более производительного аппаратного обеспечения (вертикальное масштабирование) одноканальной СМО становится слишком затратным или попросту невозможным (ввиду законов физики и научного прогресса), переход на многоканальность ради увеличения скорости обработки потока заявок — единственный выход.

Распределенная реляционная СУБД Picodata является многоканальной СМО. Ее многоканальность проявляется в сегментировании данных по множеству независимых серверов по значению ключа сегментирования.

1.3 Проблема поиска

Положим, что базовой задачей поиска в СУБД является задача поиска по значению первичного или вторичного ключа. Сегментирование данных в СУБД Picodata позволяет оптимизировать задачу поиска по ключу сегментирования. Обычно ключ сегментирования является и первичным ключом таблицы, поэтому считаем, что в нашем случае это условие тоже выполняется. Если:

- данные равномерно распределены по всем наборам реплик кластера,
- количество данных в искомой таблице равно D ,
- количество наборов реплик в кластере равно N ,
- ПО подключения к СУБД (далее — драйвер) знает функцию (далее — функция сегментирования), которая по значению первичного ключа определяет конкретный набор реплик кластера, в котором должна храниться запись,

то поиск в структуре данных, хранящей таблицу с искомой записью будет происходить среди $\frac{D}{N}$ записей, в то время, когда нераспределенным СУБД пришлось бы производить его среди полного набора записей D .

Однако, в СУБД Picodata нет решения задачи оптимизации поиска по вторичным ключам такой же эффективности. Сейчас происходит опрос кластера с поиском в локальных индексах. То есть среди всех данных D , N процессов параллельно ищут запрашиваемую запись — каждая машина совершает поиск среди $\frac{D}{N}$ записей.

Рассуждение выше наталкивает на мысль о том, что, например, если значение вторичного ключа так же однозначно определяет запись, как и первичный индекс (что нередкость), то количество совершённой бесполезной работы при подобном подходе увеличивается пропорционально количеству набору реплик кластера (N). Помимо этого, количество сетевых запросов растёт пропорционально количеству набору реплик кластера, что тоже в высокой степени влияет на быстроедействие поиска в СУБД.

1.4 Решение проблемы

Исходя из этого, было решено провести исследовательскую работу по изучению работоспособности глобального вторичного индекса (ГВИ) в СУБД Picodata. Идея заключается в сегментировании индексной структуры по значениям вторичного ключа индексируемой таблицы.

План работ следующий:

- 1) полно описать СУБД Picodata с предлагаемой индексной структурой и без как СМО;
- 2) построить математическую модель СМО для СУБД Picodata с предлагаемой индексной структурой и без: предоставить формулы расчета теоретических показателей эффективности СМО;
- 3) провести теоретический расчет показателей эффективности СМО;
- 4) разработать ПО, реализующее математическую модель в виде плагина к СУБД Picodata и тестирующее ПО для проведения экспериментов;
- 5) провести эксперименты для снятия показателей эффективности СМО на разработанном ПО при небольшом размере кластера, сравнить полученные результаты с теоретическим расчетом, объяснить расхождения, и сделать выводы — провести границы применимости распределенного индекса;
- 6) решить обратную задачу построения теоретических показателей эффективности — составить перечень рекомендаций по использованию распределенной индексной структуры.

1.5 Место и роль продукта ВКР

Основным продуктом ВКР является плагин к СУБД Picodata, добавляющий функционал глобальных индексов и интерфейс работы с ними.

Функциональные возможности плагина:

- построение глобального индекса;
- удаление глобального индекса;
- выполнение поисковых запросов с использованием глобального индекса.

Целевые пользователи плагина:

- администраторы СУБД Picodata;
- разработчики распределенных приложений.

Побочными продуктами ВКР выступают:

- 1) математическая модель индексных структур СУБД Picodata;
- 2) показатели эффективности различных индексных структур;
- 3) методические рекомендации по выбору типа индекса;
- 4) драйвер использования плагина;
- 5) результаты нагрузочного тестирования.

2 Описание предметной области

2.1 Верхнеуровневое описание СУБД Picodata

Модель функционирования СУБД Picodata представлена текстовым верхнеуровневым описанием архитектуры, некоторых особенностей и внутренних процессов.

Picodata — это распределенная СУБД, основной фокус которой является отказоустойчивое хранение сегментированных данных. Ее распределенность проявляется при соединении через сеть нескольких запущенных процессов Picodata (узлов): они собираются в кластер — единую СУБД.

Каждый кластер состоит из нескольких наборов узлов, в каждом наборе мастер-узел обрабатывает пользовательские запросы, а остальные являются его репликами. Разделение данных между наборами узлов позволяет примерно равномерно распределять нагрузку по кластеру. Например, при обновлении одной записи, обновляется только часть базы данных, расположенная на одном наборе реплик.

Picodata представляет из себя многопоточное приложение. Главным потоком является `tx_thread` — из интересующего нас, в нем происходит доступ к данным и запускается код плагинов.

При использовании резидентного движка хранения `memtx`, обновления таблицы записываются в журнал упреждающей записи, а индексы хранятся в структуре данных B^{+*} -дерево (смесь вариантов B^{+-} и B^{*-} деревьев) в ОЗУ. При построении вторичного индекса, ключом становится конкатенация значения вторичного ключа с первичным, так что записи с совпадающими вторичными ключами идут подряд. В листе дерева хранится указатель на полную запись в таблице.

Среда потока доступа к данным исполнения является асинхронной, то есть используется кооперативная многозадачность, что позволяет максимально эффективно использовать ЦПУ во время ожидания ответа дискового хранилища или сетевого интерфейса.

2.1.1 Сегментирование

Особо важно понимать механизм сегментирования в СУБД Picodata. Равномерность распределения данных, а также предотвращение лишних сетевых запросов при изменении количества наборов реплик достигается благодаря виртуальным сущностям — бакетам, которые однозначно идентифицируются натуральным числом. Количество бакетов в кластере задается единожды при запуске первого узла и не изменяется впоследствии.

Среди наборов реплик бакеты поделены поровну, например, если в кластере 3 набора реплик, а бакетов 3000, то бакеты поделены по 1000 штук (например, равными последовательными отрезками), то есть:

- первому набору реплик принадлежат бакеты с номерами от 1 по 1000;
- второму — от 1001 по 2000;
- третьему — от 2001 по 3000.

При создании сегментированной таблицы, задается набор атрибутов, называемый ключом сегментирования (обычно им же является первичный ключ) — это поля записи, на основе которых вычисляется бакет каждой записи. Номер бакета равен остатку от деления значения хеш-функции ключа сегментирования.

Любые пользовательские запросы, пришедшие на узел, преобразовываются в план исполнения — набор операций, который должен исполнить движок хранения, чтобы ответить на запрос. Если для исполнения запроса должно быть задействовано несколько наборов реплик, то на мастера каждого набора реплик отправляется план исполнения, который исполняется в локальном движке хранения, например, в случае memtx, это обход и обновление B+*-дерева в ОЗУ.

Более подробное устройство индекса, построенного на B+*-дереве можно прочитать в [1–4].

Полную документацию СУБД Picodata можно прочитать на портале документации компании [5]. Подробнее про концепты распределенных систем можно прочитать в [6].

2.2 Диаграмма «сущность-связь»

Инфологическая схема предметной области представлена в виде диаграммы «сущность-связь» на рисунке 1.

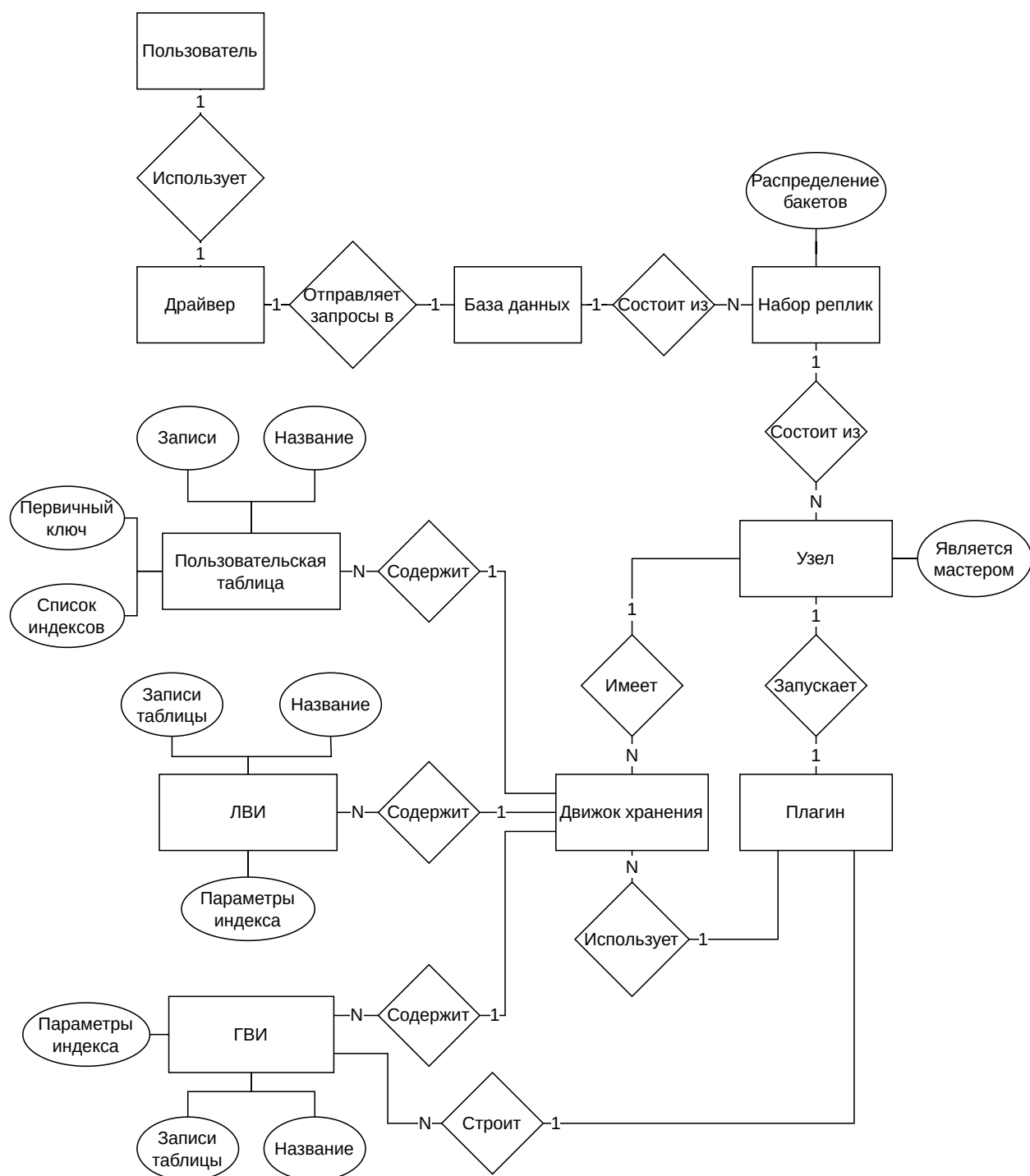


Рисунок 1 — ER-диаграмма предметной области продукта ВКР

2.2.1 Описание условных обозначений

Диаграмма выполнена в нотации Чена [7]. Используемые обозначения:

- прямоугольник — сущность;
- ромб — связь;
- овал — атрибут сущности;
- 1:1 — связь «один к одному»;
- 1:N — связь «один ко многим».

2.2.2 Описание сущностей

В модели выделены следующие сущности и их атрибуты (если они есть):

- пользователь — лицо, инициирующее запросы к СУБД;
- драйвер — ПО, предоставляющее интерфейс взаимодействия с СУБД;
- база данных — кластер СУБД Picodata;
- набор реплик — набор узлов, хранящих одинаковую часть данных, в котором один является мастером, а остальные являются репликами, принимающими и применяющими поток изменений от мастера.

Атрибуты:

- распределение бакетов — список бакетов, принадлежащих данному набор реплику;
- узел — один запущенный процесс Picodata. Атрибуты:
 - является мастером — булево значение, показывающее роль в наборе реплик: мастер или реплика;
- движок хранения — механизм хранения данных;
- пользовательская таблица — совокупность связанных данных о пользователях сервиса, хранящихся в структурированном виде.

Атрибуты:

- название — идентификатор таблицы;
- первичный ключ — список идентификаторов полей, уникально определяющих запись;

- записи — пользовательские данные;
- список индексов — набор построенных над таблицей индексов;
- ЛВИ (локальный вторичный индекс) — индексная структура, сегментированная как и индексируемая таблица. Атрибуты:
 - название — идентификатор индекса;
 - записи таблицы — проиндексированные пользовательские данные;
 - параметры индекса — значения настроек индекса: уникальность значений вторичных ключей;
- плагин — единица программного обеспечения, расширяющая функционал узла;
- ГВИ (глобальный вторичный индекс) — индексная структура, сегментированная по индексируемым полям. Атрибуты:
 - название — идентификатор индекса;
 - записи таблицы — проиндексированные пользовательские данные;
 - параметры индекса — значения настроек индекса: уникальность значений вторичных ключей;

2.2.3 Описание связей

В модели выделены следующие связи между сущностями:

- пользователь — драйвер: пользователь использует драйвер для отправления запросов к СУБД (1:1);
- драйвер — база данных: драйвер посылает запросы в базу данных (1:1);
- база данных — набор реплик: база данных состоит из наборов реплик (1:N);
- набор реплик — узел: набор реплик состоит из узлов (1:N);
- узел — движок хранения: узел имеет несколько движков хранения (1:N);
- движок хранения — пользовательская таблица: движок хранения содержит множество пользовательских таблиц (1:N);

- движок хранения — ЛВИ: движок хранения содержит множество локальных вторичных индексов (1:N);
- движок хранения — ГВИ: движок хранения содержит множество глобальных вторичных индексов (1:N);
- узел — плагин: узел запускает плагин построения глобальных вторичных индексов (1:1);
- плагин — движок хранения: плагин использует движки хранения для построения глобальных вторичных индексов (1:N);
- плагин — ГВИ: плагин строит множество глобальных вторичных индексов (1:N).

3 Математическая модель функционирования СУБД Picodata как СМО

В данной главе построена математическая модель СУБД Picodata, основываясь на теории массового обслуживания. Рассматриваются локальный вторичный индекс (ЛВИ) — единственная индексная структура, реализованная в Picodata на данный момент, и новая, предлагаемая структура — глобальный вторичный индекс (ГВИ) в контексте обновления данных и поиска по ним.

Это необходимо для математического обоснования создания альтернативного алгоритма индексирования данных в распределенной СУБД Picodata.

В качестве материала для изучения теории массового обслуживания были использованы [8–10].

3.1 СУБД Picodata как СМО

В контексте работы с индексами в СУБД Picodata, удобно разделить каждый узел на две роли — координатора и исполнителя. Такое разделение приведено сугубо для удобства расчетов математической модели, на деле же все узлы являются одним и тем же приложением с идентичной логикой. Роль узла «зафиксирована» только на время обработки одного запроса и зависит исключительно от выбора пользователем узла, который примет запрос, то есть координатора.

Алгоритм работы координатора представлен следующим циклом:

- 1) принять запрос пользователя;
- 2) составить планы исполнения для мастеров нужных наборов реплик (исполнителей);
- 3) разослать планы исполнения исполнителям;
- 4) дождаться результата от исполнителей;
- 5) объединить полученные данные;
- 6) отправить пользователю запрашиваемые данные.

Работа исполнителя происходит между пунктами 3 и 4 работы координатора и ее алгоритм представлен следующим циклом:

- 1) принять план исполнения от координатора;
- 2) исполнить план с помощью движка хранения на своей части данных;
- 3) отправить результат исполнения обратно на координатора.

Получается координатор «управляет» всем кластером на время запроса, а исполнитель — только локальными структурами данных. Координатор бьет изначальный запрос на подзадачи, которые решают несколько исполнителей.

В качестве СМО будет описан именно исполнитель, так как работа координатора занимает малое время. Это будет показано в пункте 3.3.

Чтобы избежать излишней сложности модели, постановим, что каждый набор реплик кластера состоит из одного узла. Это не влияет на точность модели, так как реплики являются резервным хранилищем, на котором не исполняются запросы.

В таблице 1 приведена характеристика узла СУБД Picodata как СМО. Положения таблицы верны для обеих индексных структур и являются справедливыми и для поиска, и для обновления:

Таблица 1 — Характеристика роли координатора узла в СУБД Picodata как СМО

Элемент СМО	Элемент СУБД
Канал обслуживания	tx_thread
Поток заявок	Запросы на поиск или обновление данных сегментированной таблицы
Поток ответов	Отфильтрованные данные таблицы в случае поиска и подтверждение операции в случае обновления таблицы
Дисциплина очереди	Очередь бесконечной длины с ограничением по времени ожидания (таймаут заявки)

Для простоты модели, допустим, что в течение работы количество заявок является случайной величиной следующей закону распределения Пуассона (время между заявками следует экспоненциальному закону распределения) с параметром λ , причем этот поток обладает свойствами:

- стационарности — число заявок не зависит от времени начала работы СМО;
- ординарности — в бесконечно малый промежуток времени шанс получения нескольких заявок стремится к нулю;
- является потоком без последствия — количество заявок в настоящем не зависит от количества заявок в прошлом.

Более подробное описание этих свойств можно найти в [9].

Для расчета задержки сетевых запросов, будем использовать случайную величину с гамма-распределением согласно исследованиям [11,12].

В контексте поиска по индексу, элементарной задачей является нахождение записей с фильтрацией по значению одного вторичного ключа.

В контексте обновления индекса, элементарной задачей является обновление одной записи во вторичном индексе.

Для расчета времени работы узла, будем ориентироваться на количество операций сравнения ключей при поиске и обновлении в B^+ -дереве, деленное на частоту проведения этих операций ЦПУ и количество операций загрузки данных по указателям, деленное на частоту загрузки блока памяти из запоминающего устройства.

Также, будем учитывать время, потраченное на запись в журнал упреждающей записи, и скажем, что оно константное, так как представляет из себя просто добавление байтов в конец файла, указатель на конец которого известен в момент записи.

Итого, кластер СУБД Picodata моделируется как N независимых (из-за различных наборов данных) СМО $M/G/1$ согласно нотации приведенной в [10], где N — это количество наборов реплик в кластере.

3.1.1 Показатели эффективности

Возьмем следующие показатели эффективности СМО:

- максимальное количество запросов в секунду;
- вероятность таймаута заявки;
- среднее время ожидания обслуживания.

Для построения расчетов показателей эффективности используются следующие параметры:

- количество наборов реплик в кластере N ;
- объем пользовательских данных D (количество записей в таблице);
- количество уникальных вторичных ключей (кардинальность множества вторичных ключей) K ;
- размер одной записи r (в байтах);
- частота операций сравнения в секунду на ЦПУ f_{cpu} ;
- частота операций загрузки и записи данных по указателю из устройства памяти f_{mem} ;
- порядок B⁺-дерева m ;
- время таймаута T_{timeout} в секундах;
- скорость сети между пользователем и кластером СУБД v_{user} в байтах в секунду;
- скорость сети между узлами v_{cluster} в байтах в секунду;
- значения параметров случайных величин:
 - λ_{search} и λ_{update} — интенсивности потоков заявок (заявок в секунду) на поиск и обновление соответственно;
 - k_{user} и θ_{user} — параметры гамма-распределения $\Gamma(k_{\text{user}}, \theta_{\text{user}})$ времени сетевой задержки между пользователем и кластером СУБД. Математическое ожидание такого распределения равно $k_{\text{user}}\theta_{\text{user}}$, дисперсия равна $k_{\text{user}}\theta_{\text{user}}^2$;
 - k_{cluster} и θ_{cluster} — параметры гамма-распределения $\Gamma(k_{\text{cluster}}, \theta_{\text{cluster}})$ времени сетевой задержки между узлами СУБД. Математическое ожидание такого распределения равно $k_{\text{cluster}}\theta_{\text{cluster}}$, дисперсия равна $k_{\text{cluster}}\theta_{\text{cluster}}^2$.

3.2 Общие положения модели

Для обеих индексных структур и обоих видов запросов взаимодействие пользователя на уровне кластера выглядит одинаково: координатор принимает запрос пользователя, по некому алгоритму собирает данные со всех исполнителей и возвращает их пользователю. Для удобства, расчет времени этого взаимодействия вынесено в лемму 1.

Время ожидания пользователя T^{user} исполнения заявки является случайной величиной и рассчитывается по следующей формуле:

$$T^{\text{user}} = t_{\text{user}}^{\text{request}} + \frac{Q}{v_{\text{user}}} + W^{\text{coordinate}} + t_{\text{user}}^{\text{response}} + \frac{R}{v_{\text{user}}}, \quad (1)$$

где:

- $t_{\text{user}}^{\text{request}}$ — время задержки передачи запроса от пользователя к СУБД по сети;
- Q — размер пользовательского запроса;
- v_{user} — скорость передачи данных между кластером СУБД и пользователем;
- $W^{\text{coordinate}}$ — время ожидания обработки запроса координатором;
- $t_{\text{user}}^{\text{response}}$ — время задержки передачи результата от СУБД к пользователю по сети;
- R — размер результата запроса.

lemma 1 — О времени ожидания пользователя

Взаимодействие пользователя с кластером начинается с отправки запроса по сети.

Задержку сети между пользователем и СУБД, то есть время между отправлением первого байта пользователем и его получением СУБД, мы моделируем как гамма-распределение, согласно рассуждениям в описании координатора, обозначим ее как $t_{\text{user}}^{\text{request}} \sim \Gamma(k_{\text{user}}, \theta_{\text{user}})$.

Передача по сети запроса, весящего Q байтов со скоростью v_{user} байтов в секунду занимает Q/v_{user} секунд.

Положим, что время ожидание в очереди и обработки запроса координатором занимает $W^{\text{coordinate}}$ секунд.

После получения результата в СУБД, он должен быть отправлен обратно пользователю по сети. Задержку сети между СУБД и пользователем моделируем так же — обозначим ее как $t_{\text{user}}^{\text{response}} \sim \Gamma(k_{\text{user}}, \theta_{\text{user}})$.

Передача по сети результата, весящего R байтов со скоростью v_{user} байтов в секунду занимает R/v_{user} секунд.

При получении результата, взаимодействие пользователя с кластером кончается.

Итого, время ожидания пользователя, которое мы обозначим T^{user} является суммой вышеупомянутых величин:

$$T^{\text{user}} = t_{\text{user}}^{\text{request}} + \frac{Q}{v_{\text{user}}} + W^{\text{coordinate}} + t_{\text{user}}^{\text{response}} + \frac{R}{v_{\text{user}}}. \quad (2)$$

Часть суммы — случайная величина, значит и вся сумма — случайная величина.

В пунктах 3.3, 3.4, 3.5 и 3.6 рассматриваются алгоритмы конкретных индексных структур и запросов. Переменные, специфичные индексной структуре и запросу будут помечены в нижнем индексе. То есть, например, время ожидания исполнения запроса координатора при поиске в ЛВИ будет обозначаться $W_{\text{search,LSI}}^{\text{coordinate}}$, а время ожидания пользователем обновления ГВИ будет обозначаться $T_{\text{update,GSI}}^{\text{user}}$.

3.3 Поиск в локальном индексе

Локальные индексы устроены следующим образом:

- как сказано в пункте 2.1, данные сегментированных таблиц распределены примерно равномерно среди наборов реплик, а принадлежность записи к набору реплик определяется значением ключа сегментирования;
- часть индекса находится на каждом наборе реплик, причем данные индекса поделены по тому же ключу сегментирования, что и индексируемая таблица.

Алгоритм поиска в кластере с такой индексной структурой выглядит следующим образом:

- 1) координатор принимает пользовательский запрос, составляет план локального поиска
- 2) координатор отправляет план локального запроса всем исполнителем (мастеру каждого набора реплик);
- 3) на каждом исполнителе происходит поиск записей по значению вторичного ключа в B^{+} -дереве части индекса;
- 4) исполнитель отправляет найденные записи координатору;
- 5) координатор ждет ответов от всех наборов реплик, объединяет их и отправляет найденные записи пользователю.

Время ожидания исполнения запроса координатором $W_{\text{search,LSI}}^{\text{coordinate}}$ является случайной величиной и рассчитывается по следующей формуле:

$$\begin{aligned}
 W_{\text{search,LSI}}^{\text{coordinate}} &= W_{\text{search,q,LSI}}^{\text{execute}} + T_{\text{search,LSI}}^{\text{coordinate}} = \\
 &= W_{\text{search,q,LSI}}^{\text{execute}} + \max_{i=1..N} [T_{\text{search,LSI},i}^{\text{coordinate}}] = W_{\text{search,q,LSI}}^{\text{execute}} + \\
 &+ \max_{i=1..N} \left[t_{\text{replicaset}}^{\text{request},i} + \frac{P}{v_{\text{cluster}}} + W_{\text{search,LSI}}^{\text{execute}} + t_{\text{replicaset}}^{\text{response},i} + \frac{sr}{v_{\text{cluster}}} \right],
 \end{aligned} \tag{3}$$

где:

- $W_{\text{search,q,LSI}}^{\text{execute}}$ — время ожидания запроса в очереди исполнителя;
- $T_{\text{search,LSI},i}^{\text{coordinate}}$ — время ожидания ответа от исполнителя под номером i ;
- $t_{\text{replicaset}}^{\text{request},i}$ и $t_{\text{replicaset}}^{\text{response},i}$ — времена задержки передачи запроса между координатором и исполнителем под номером i по сети;
- P — размер плана исполнения;
- v_{cluster} — скорость передачи данных между узлами СУБД;
- $W_{\text{search,LSI}}^{\text{execute}}$ — время ожидания обработки запроса исполнителем;
- s — количество записей с искомым вторичным ключом на наборе реплик;
- r — размер одной записи.

theorem 3 — О времени ожидания исполнения запроса координатором при поиске в ЛВИ

Для начала рассмотрим время, которое проводится исключительно на самом координаторе.

Запрос сначала попадает в очередь исполнителя (так как координатор и исполнитель — это одно и то же приложение, то и очередь у них одна) и находится там $W_{\text{search},q,\text{LSI}}^{\text{execute}}$ секунд.

После этого происходит составление плана запроса, время которого не учитывается, так как на практике занимает чуть больше 6 микросекунд на ЦПУ игрового ноутбука со скоростью 4.46 ГГц, что на 4 порядка меньше чем среднее время пинга домашнего роутера с передачей 56 байтов, подключенного по Wi-Fi — 23 миллисекунды. Пинг домашнего роутера занимает порядка десяти миллисекунд, так что можно предположить, что передача сообщений с результатами между узлами будет занимать не меньше времени, особенно если узлы расположены отдаленно географически.

Итого, на координаторе заявка обрабатывается $W_{\text{search},q,\text{LSI}}^{\text{execute}}$ секунд, далее он делегирует работу исполнителям.

Рассмотрим сколько времени занимает обработка поиска на исполнителях.

Составленный план запроса отправляется каждому исполнителю по сети, которых всего N штук.

Каждое отправление плана сопровождается сетевой задержкой, которая длится $t_{\text{replicaset}}^{\text{request},i}$, моделируемая как гамма-распределение $\Gamma(k_{\text{cluster}}, \theta_{\text{cluster}})$, аналогично лемме 1.

Положим, что план запроса весит P байтов, а скорость передачи данных между узлами кластера равна v_{cluster} байтов в секунду. Тогда весь план будет передан через P/v_{cluster} секунд.

На исполнителе время ожидания исполнения заявки, то есть время, проведенное в очереди в сумме с временем непосредственной обработки заявки обозначим как $W_{\text{search},\text{LSI}}^{\text{execute}}$.

Результат работы каждого исполнителя отправляется координатору отдельным сетевым запросом с задержкой $t_{\text{replicaset}}^{\text{request},i}$, моделируемой гамма-распределением $\Gamma(k_{\text{cluster}}, \theta_{\text{cluster}})$.

Исходя из предположения, что данные распределены равномерно (в том числе по вторичным ключам) по наборам реплик, то на каждом исполнителе находится s , равное $\frac{D}{NK}$ искомых записей.

Заметим, что каждая заявка на поиск порождает N заявок исполнителям, причем интенсивность потока исполнителя так же равна λ_{search} . Интенсивность потока во всем кластере суммарно равна $\lambda_{\text{search}} N$.

Все параметры, необходимые для расчета, кроме времени ожидания обработки подзадачи исполнителем $W_{\text{search,LSI}}^{\text{execute}}$ и времени ожидания в очереди исполнителя $W_{\text{search,q,LSI}}^{\text{execute}}$ известны. Для нахождения обеих величин необходимо вычислить время обработки заявки исполнителем $T_{\text{search,LSI}}^{\text{execute}}$.

Чтобы не усложнять расчеты случайностью всех процессов поиска в $B+^*$ -дереве, будет рассмотрен худший случай поиска. Так что $T_{\text{search,LSI}}^{\text{coordinate}}$ — это скалярная величина.

На каждом исполнителе хранится по $d = \frac{D}{N}$ записей, так что поиск первой записи по вторичному ключу в $B+^*$ -дереве требует $\log_m d$ операций перехода по указателям и $\log_m d \cdot \log_2(2m - 1)$ операций сравнения ключей.

Если рассмотреть случай, когда искомый ключ является самым правым ключом листа, при этом все листья правее имеют $\frac{2}{3}(2m - 1)$ ключей (минимальное количество ключей в узле в соответствии с определением $B+^*$ -деревя), получаем, что чтение s записей из листовых узлов $B+^*$ -деревя требует максимум $\frac{s-1}{\frac{2}{3}(2m-1)}$ дополнительных переходов по указателям между листьями. После нахождения искомого ключа нужно перейти по указателю на полную запись в таблице.

Итого, время решения подзадачи поиска равно:

$$T_{\text{search,LSI}}^{\text{execute}} = \frac{\log_m d \cdot \log_2(2m - 1)}{f_{\text{cpu}}} + \frac{\log_m d + 3 \cdot \frac{s-1}{4m-2} + 1}{f_{\text{mem}}}. \quad (6)$$

Теперь можно найти время ожидания в очереди исполнителя $W_{\text{search,q,LSI}}^{\text{execute}}$ и время работы над заявкой исполнителем $W_{\text{search,LSI}}^{\text{execute}}$. Для этого воспользуемся формулой Полячека-Хинчина [10].

Загрузка исполнителя равна:

$$\rho_{\text{search,LSI}}^{\text{executor}} = \lambda_{\text{search}} \cdot T_{\text{search,LSI}}^{\text{execute}}, \quad (7)$$

значит он будет справляться с нагрузкой при $\rho_{\text{search,LSI}}^{\text{executor}} < 1$.

По формуле Полячека—Хинчина рассчитаем среднее количество заявок в исполнителе (количество заявок в очереди и ныне обслуживающихся):

$$\begin{aligned} L_{\text{search,LSI}}^{\text{execute}} &= \rho_{\text{search,LSI}}^{\text{executor}} + \frac{\rho_{\text{search,LSI}}^{\text{executor}^2} + \lambda_{\text{search}} D[T_{\text{search,LSI}}^{\text{execute}}]}{2(1 - \rho_{\text{search,LSI}}^{\text{executor}})} = \\ &= \rho_{\text{search,LSI}}^{\text{executor}} + \frac{\rho_{\text{search,LSI}}^{\text{executor}^2}}{2(1 - \rho_{\text{search,LSI}}^{\text{executor}})}, \end{aligned} \quad (8)$$

и среднее время ожидания заявки в исполнителе в соответствии с законом Литтла, приведенным в [9,10]:

$$W_{\text{search,LSI}}^{\text{execute}} = \frac{L_{\text{search,LSI}}^{\text{execute}}}{\lambda_{\text{search}}} = T_{\text{search,LSI}}^{\text{execute}} + \frac{\rho_{\text{search,LSI}}^{\text{executor}} T_{\text{search,LSI}}^{\text{execute}}}{2(1 - \rho_{\text{search,LSI}}^{\text{executor}})}. \quad (9)$$

Время, проведенное в очереди равно времени проведенному в системе без времени обслуживания заявки, тогда:

$$W_{\text{search,q,LSI}}^{\text{execute}} = W_{\text{search,LSI}}^{\text{execute}} - T_{\text{search,LSI}}^{\text{execute}} = \frac{\rho_{\text{search,LSI}}^{\text{executor}} T_{\text{search,LSI}}^{\text{execute}}}{2(1 - \rho_{\text{search,LSI}}^{\text{executor}})}. \quad (10)$$

Оценим время работы координатора, связанное с одним исполнителем:

$$T_{\text{search,LSI},i}^{\text{coordinate}} = t_{\text{replicaset}}^{\text{request},i} + \frac{P}{v_{\text{cluster}}} + W_{\text{search,LSI}}^{\text{execute}} + t_{\text{replicaset}}^{\text{response},i} + \frac{sr}{v_{\text{cluster}}}. \quad (11)$$

Только времена сетевой задержки ($t_{\text{replicaset}}^{\text{request},i}$, $t_{\text{replicaset}}^{\text{response},i}$) — случайные величины, остальные — скалярные, так что, согласно свойствам гамма-распределения, приведенным в [13], время работы координатора на одном наборе реплик — случайная величина, имеющая сдвинутое гамма-распределение. Случайную часть $T_{\text{search,LSI},i}^{\text{coordinate}}$ обозначим Y , а детерминированную — $C^{\text{coordinate}}$:

$$C^{\text{coordinate}} = W_{\text{search,LSI}}^{\text{execute}} + \frac{P + sr}{v_{\text{cluster}}}, \quad (12)$$

$$Y = T_{\text{search,LSI},i}^{\text{coordinate}} - C^{\text{coordinate}} \sim \Gamma(2k_{\text{cluster}}, \theta_{\text{cluster}}). \quad (13)$$

Следовательно функция распределения равна:

$$F_Y(t) = P(Y \leq t) = \frac{\gamma(2k_{\text{cluster}}, t/\theta_{\text{cluster}})}{\Gamma(2k_{\text{cluster}})}. \quad (14)$$

Согласно формуле (3), время работы координатора равно:

$$T_{\text{search,LSI}}^{\text{coordinate}} = \max_{i=1..N} T_{\text{search,LSI},i}^{\text{coordinate}}, \quad (15)$$

случайную часть $T_{\text{search,LSI}}^{\text{coordinate}}$ обозначим Z , максимум среди одинаковых неслучайных величин $C^{\text{coordinate}}$ равен самому себе:

$$Z = T_{\text{search,LSI}}^{\text{coordinate}} - C^{\text{coordinate}} \quad (16)$$

значит, функция распределения Z равна:

$$F_Z(t) = [F_Y(t)]^N = \left[\frac{\gamma(2k_{\text{cluster}}, t/\theta_{\text{cluster}})}{\Gamma(2k_{\text{cluster}})} \right]^N, \quad (17)$$

что позволяет нам численно найти моменты $T_{\text{search,LSI}}^{\text{coordinate}}$ через функцию выживания [14]:

$$M[Z] = \int_0^\infty \left(1 - \left[\frac{\gamma(2k_{\text{cluster}}, t/\theta_{\text{cluster}})}{\Gamma(2k_{\text{cluster}})} \right]^N \right) dt, \quad (18)$$

$$M[Z^2] = \int_0^\infty 2t \left(1 - \left[\frac{\gamma(2k_{\text{cluster}}, t/\theta_{\text{cluster}})}{\Gamma(2k_{\text{cluster}})} \right]^N \right) dt, \quad (19)$$

$$M[T_{\text{search,LSI}}^{\text{coordinate}}] = M[Z] + C^{\text{coordinate}}, \quad (20)$$

$$D[T_{\text{search,LSI}}^{\text{coordinate}}] = D[Z] = M[Z^2] - (M[Z])^2. \quad (21)$$

В свою очередь:

$$M[W_{\text{search,LSI}}^{\text{coordinate}}] = W_{\text{search,q,LSI}}^{\text{execute}} + M[T_{\text{search,LSI}}^{\text{coordinate}}], \quad (22)$$

$$D[W_{\text{search,LSI}}^{\text{coordinate}}] = D[T_{\text{search,LSI}}^{\text{coordinate}}] \quad (23)$$

Случайную часть времени ожидания пользователя $T_{\text{search,LSI}}^{\text{user}}$, определенное в формуле (1), обозначим V , тогда:

$$V = t_{\text{user}}^{\text{request}} + t_{\text{user}}^{\text{response}} \sim \Gamma(2k_{\text{user}}, \theta_{\text{user}}), \quad (24)$$

из чего можно найти математическое ожидание и дисперсию времени ожидания пользователя:

$$\begin{aligned} M[T_{\text{search,LSI}}^{\text{user}}] &= M[V] + M[W_{\text{search,LSI}}^{\text{coordinate}}] + \frac{Sr}{v_{\text{user}}} = \\ &= k_{\text{user}} \theta_{\text{user}} + M[W_{\text{search,LSI}}^{\text{coordinate}}] + \frac{Sr}{v_{\text{user}}}, \end{aligned} \quad (25)$$

$$\begin{aligned} D[T_{\text{search,LSI}}^{\text{user}}] &= D[V] + D[W_{\text{search,LSI}}^{\text{coordinate}}] = \\ &= k_{\text{user}} \theta_{\text{user}}^2 + D[W_{\text{search,LSI}}^{\text{coordinate}}]. \end{aligned} \quad (26)$$

3.3.1 Расчет показателей эффективности

Максимальное количество запросов в секунду достигается при максимальной загрузке системы. Так как мы договорились, что составление плана исполнения координатором не учитывается в модели, на него можно подать бесконечную нагрузку. Тогда единственным ограничивающим фактором системы является время обработки заявки исполнителем $T_{\text{search,LSI}}^{\text{execute}}$.

Для устойчивости системы, нагрузка системы $\rho_{\text{search,LSI}}^{\text{executor}}$ должна не превышать единицу. По определению, нагрузка системы рассчитывается по формуле (7), значит максимальная нагрузка системы рассчитывается по следующей формуле:

$$\lambda_{\text{search}}^{\text{max}} = \frac{1}{T_{\text{search,LSI}}^{\text{execute}}}. \quad (27)$$

Для вычисления вероятности необслуживания заявки (таймаут заявки), нужно рассчитать вероятность того, что время обслуживания превысит максимальное время обслуживания заявки, то есть рассчитать следующее выражение:

$$P(T_{\text{search,LSI}}^{\text{user}} \geq T_{\text{timeout}}) \quad (28)$$

3.4 Поиск в глобальном индексе

3.4.1 Расчет показателей эффективности

3.5 Обновление в локальном индексе

Алгоритм записи в кластере с такой индексной структурой выглядит следующим образом:

- 1) координатор принимает пользовательский запрос, рассчитывает бакет обновляемой записи и отправляет план исполнения соответствующему исполнителю;
- 2) исполнитель добавляет информацию об обновлении таблицы в журнал упреждающей записи, обновляет В+*-дерево индекса первичного ключа и обновляет В+*-дерево вторичного индекса;
- 3) исполнитель отправляет подтверждение записи координатору;
- 4) координатор отправляет подтверждение записи пользователю.

Получается, каждая заявка на обновление порождает ровно одну подзадачу для одного исполнителя. Поток запросов координатора λ_{update} распределяется равномерно на исполнителей: $\lambda_{\text{update}}^{(\text{sub})} = \frac{\lambda_{\text{update}}}{N}$.

Обновление начинается с записи в журнал упреждающей записи, которое занимает константное время T_{WAL} . После этого происходит поиск позиции ключа в В+*-дереве для обновления, на что тратится $\log_m d \cdot \log_2(2m - 1)$ операций сравнения ключей и $\log_m d$ переходов по указателям.

После поиска происходит обновление дерева, например вставка нового элемента. В лучшем случае, когда лист не заполнен до конца, вставка происходит за одну операцию записи в запоминающее устройство, в худшем — все узлы в пути до листа (включая сам лист) нужно разделить на два. Для простоты модели, сошлемся на статью [15], в которой доказано, что средняя заполненность В*-дерева, а значит и средняя заполненность каждого узла равна $P_{\text{full}} = 0.81$, где 0 — узел пуст, 1 — узел полон. Получается, что в среднем остается $(2m - 1)(1 - 0.81) = (2m - 1)0.19$ вставок перед переполнением узла, соответственно вероятность переполнения на конкретной вставке равна $((2m - 1)0.19)^{-1} \approx \frac{5.26}{2m-1}$. Разделение узла происходит за 3 операции записи — перезапись разделенного узла, запись нового братского узла и перезапись родительского узла. Случай каскадного

разделения родительских узлов возможен, но маловероятен (вероятность этого события уменьшается с каждым родителем экспоненциально), так что он не учтен.

Итого, время решения подзадачи обновления равно:

$$T_{\text{update,LSI}}^{\text{execute}} = T_{\text{WAL}} + \frac{\log_m d \cdot \log_2(2m - 1)}{f_{\text{cpu}}} + \frac{\log_m d + 3 \cdot \frac{5.26}{2m-1}}{f_{\text{mem}}} \quad (29)$$

Это тоже детерминированная величина.

3.6 Обновление в глобальном индексе

3.7 Результат расчетов и сравнение

4 Модель функционирования плагина распределенного индекса

4.1 Контекстная диаграмма

4.2 Диаграмма потоков данных

4.2.1 Функциональная декомпозиция

5 Архитектура плагина распределенного индекса

5.1 Архитектура плагина

5.1.1 Диаграмма развертывания

5.1.2 Представление данных

5.1.3 Описание алгоритмов

6 Описание способов определения показателей качества ПО

6.1 Показатели качества

6.2 Способы определения показателей качества

7 Программа и методики испытаний ПО

7.1 Модель функционирования системы тестирования

7.1.1 Контекстная диаграмма

7.1.2 Диаграмма потоков данных

7.1.2.1 Функциональная декомпозиция

7.2 Архитектура системы тестирования индексных структур

7.2.1 Общие сведения

7.2.2 Структурные представления

7.2.3 Архитектура решения

7.2.3.1 Основные положения архитектуры

7.2.3.2 Диаграмма развертывания

7.2.3.3 Представление данных

7.3 Сравнение полученных результатов с математическими моделями

ЗАКЛЮЧЕНИЕ

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Википедия. В+-дерево [Электронный ресурс]. 2026. URL: <https://ru.wikipedia.org/wiki/B%E2%81%BA-%D0%B4%D0%B5%D1%80%D0%B5%D0%B2%D0%BE>.
2. Википедия. В*-дерево [Электронный ресурс]. 2026. URL: https://ru.wikipedia.org/wiki/B*-%D0%B4%D0%B5%D1%80%D0%B5%D0%B2%D0%BE.
3. Tarantool Developers. Tarantool source code: bps_tree.h [Электронный ресурс]. 2025. URL: https://github.com/tarantool/tarantool/blob/4e27591f685e9c695853a0165d4081148d12e315/src/lib/salad/bps_tree.h (дата обращения: 11.04.2026).
4. Бабин О. Как написать свой индекс в Tarantool [Электронный ресурс]. 2020. URL: <https://habr.com/ru/companies/vk/articles/505880/>.
5. Портал документации Picodata [Электронный ресурс]. ООО "Пикодата", 2026. URL: <https://docs.picodata.io/picodata/stable/> (дата обращения: 02.03.2026).
6. Kleppmann M. Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems. Sebastopol, CA: O'Reilly Media, 2017.
7. Chen P. P.-S. The entity-relationship model—toward a unified view of data // ACM Transactions on Database Systems. Association for Computing Machinery (ACM), 1976. Т. 1, № 1. Сс. 9–36.
8. Вентцель Е. С. Исследование операций: задачи, принципы, методология. 2-е изд. "Наука" Главная редакция физико-математической литературы, 1988.
9. Розенберг В. Я., Прохоров А. И. Что такое теория массового обслуживания. 2-е изд. Москва: Советское радио, 1965. С. 256.
10. Shortle J. F. и др. Fundamentals of Queueing Theory. 5-е изд. Hoboken, NJ: Wiley, 2018.
11. Liu Y., Li J., Gu N. Modeling Internet Link Delay Based on Measurement // 2009 International Conference on Computational Science and Engineering. Vancouver, BC, Canada: IEEE, 2009. Т. 4. Сс. 874–879.

12. Valadão E. D. и др. Caracterização de tempos de ida-e-volta na Internet // Revista de Informática Teórica e Aplicada. Porto Alegre, Brazil: UFRGS, 2012. Т. 19, № 2. Сс. 4–20.
13. Кибзун А. И., Горяинова Е. Р., Наумов А. В. Теория вероятностей и математическая статистика. Базовый курс с примерами и задачами. 3-е изд. Москва: Физматлит, 2011.
14. Ross S. M. A First Course in Probability. 10-е изд. Pearson, 2019.
15. Leung C. H. C. Approximate storage utilisation of B-trees: A simple derivation and generalisations // Information Processing Letters. Elsevier, 1984. Т. 19, № 4. Сс. 199–201.